Customer Shopping Behavior Analysis Project Report

1. Introduction

Customer shopping behavior analysis helps businesses understand how customers purchase products, what influences their buying decisions, and how to improve sales strategies.
In this project, we analyze customer transaction data to identify patterns, trends, and key factors affecting purchasing behavior using Python, SQL, and Power BI.

2. Objectives

- Analyse customer purchasing patterns
- Identify high-value customers
- Understand product preferences
- Improve sales and marketing strategies
- Build an interactive dashboard

3. Tools & Technologies Used

Python (Pandas, NumPy)

SQL (MySQL)

Power BI

4. Dataset collected

Dataset collected through Kaggle.com

5. Data Preprocessing (Python)

Step 1: import libraries and load the dataset

```
]: import pandas as pd
   import matplotlib.pyplot as plt
   import seaborn as sns
   df = pd.read_csv("customer_shopping_behavior.csv")
```

Step 2: Understand the dataset

```
df.head()
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Shipping Type	Discount Applied	Promo Code Used	Previous Purchases	Pay M
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Express	Yes	Yes	14	V
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Express	Yes	Yes	2	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Free Shipping	Yes	Yes	23	
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon	Spring	3.5	Yes	Next Day Air	Yes	Yes	49	
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise	Spring	2.7	Yes	Free Shipping	Yes	Yes	31	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer ID             3900 non-null   int64
 1   Age                     3900 non-null   int64
 2   Gender                  3900 non-null   object
 3   Item Purchased          3900 non-null   object
 4   Category                3900 non-null   object
 5   Purchase Amount (USD)   3900 non-null   int64
 6   Location                3900 non-null   object
 7   Size                    3900 non-null   object
 8   Color                   3900 non-null   object
 9   Season                  3900 non-null   object
 10  Review Rating           3863 non-null   float64
 11  Subscription Status     3900 non-null   object
 12  Shipping Type           3900 non-null   object
 13  Discount Applied        3900 non-null   object
 14  Promo Code Used         3900 non-null   object
 15  Previous Purchases      3900 non-null   int64
 16  Payment Method          3900 non-null   object
 17  Frequency of Purchases  3900 non-null   object
dtypes: float64(1), int64(4), object(13)
memory usage: 548.6+ KB
```

Step 3: Data cleaning

```
df.isna().sum()
```

```
Customer ID               0
Age                       0
Gender                    0
Item Purchased            0
Category                  0
Purchase Amount (USD)     0
Location                  0
Size                      0
Color                     0
Season                    0
Review Rating            37
Subscription Status       0
Shipping Type             0
Discount Applied          0
Promo Code Used           0
Previous Purchases        0
Payment Method            0
Frequency of Purchases    0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

1: Replacing null values of Review Rating with median values

```python
df.duplicated().sum()
```

```
np.int64(0)
```

```python
df[df["Review Rating"].isna()].index
```

```
Index([  38,   50,   80,   96,  262,  330,  356,  383,  419,  442,  452,  460,
        491,  512,  560,  628,  799,  829,  861,  888, 1072, 1109, 1124, 1134,
       1151, 1176, 1189, 1208, 1220, 1230, 1247, 1292, 1313, 1326, 1334, 1341,
       1370],
      dtype='int64')
```

```python
missing_rows = df["Review Rating"].isna()
df["Review Rating"]=df.groupby("Category")["Review Rating"].transform(lambda x: x.fillna(x.median()))
```

```python
df.loc[38,"Review Rating"]
```

2: Standardize the column headers

```python
df.columns = df.columns.str.strip().str.lower().str.replace(" ","_")
df.rename(columns = {"purchase_amount_(usd)" : "purchase_amount"},inplace = True)
df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'promo_code_used', 'previous_purchases',
       'payment_method', 'frequency_of_purchases'],
      dtype='object')
```

3. Feature Engineering

Create new columns like:

```python
# create a column called "age_group"
labels = ["Young Adult","Adult","Middle-aged","Senior"]
df["age_group"] = pd.qcut(df["age"],q=4,labels=labels)
df[["age","age_group"]]
```

	age	age_group
0	55	Middle-aged
1	19	Young Adult
2	50	Middle-aged
3	21	Young Adult
4	45	Middle-aged
...	...	...
3895	40	Adult
3896	52	Middle-aged
3897	46	Middle-aged
3898	44	Adult
3899	52	Middle-aged

3900 rows × 2 columns

```python
# create a column purchase_frequency_days
frequency_mapping = {
    "Fortnightly" : 14,
    "Weekly" : 7,
    "Monthly" : 30,
    "Quarterly" : 90,
    "Bi-Weekly" : 14,
    "Annually" : 365,
    "Every 3 Months" : 90
}
df["purchase_frequency_days"] = df["frequency_of_purchases"].map(frequency_mapping)
df[["frequency_of_purchases","purchase_frequency_days"]].head(5)
```

	frequency_of_purchases	purchase_frequency_days
0	Fortnightly	14
1	Fortnightly	14
2	Weekly	7
3	Weekly	7
4	Annually	365

4. Dropping column that are same

```python
(df["discount_applied"] == df["promo_code_used"]).all()
```

```
np.True_
```

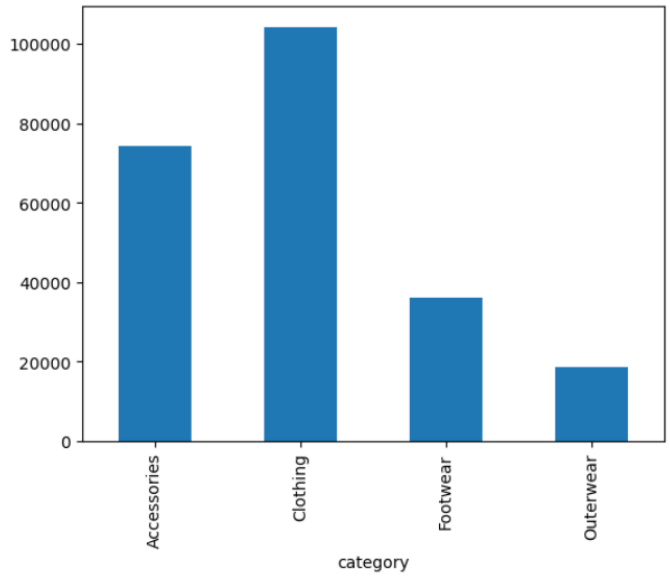
```python
df = df.drop("promo_code_used",axis = 1)
df.columns
```

```
Index(['customer_id', 'age', 'gender', 'item_purchased', 'category',
       'purchase_amount', 'location', 'size', 'color', 'season',
       'review_rating', 'subscription_status', 'shipping_type',
       'discount_applied', 'previous_purchases', 'payment_method',
       'frequency_of_purchases', 'age_group', 'purchase_frequency_days'],
      dtype='object')
```

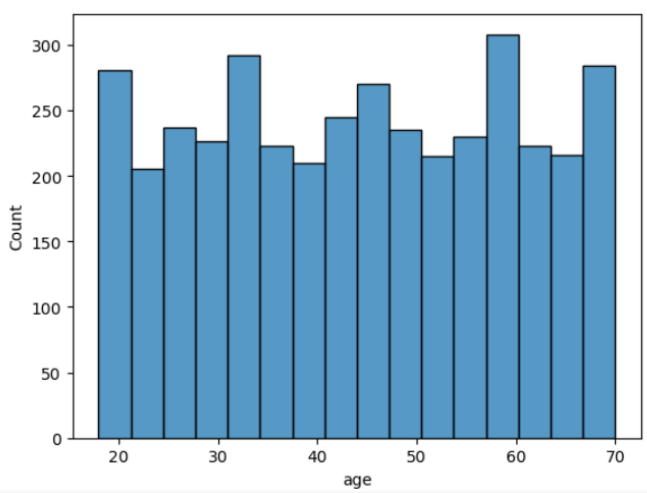
6. Exploratory Data Analysis (Python)

```
# Exploratory Data Analysis
# Total Sales by Category
df.groupby("category")["purchase_amount"].sum().plot(kind='bar')
```

```
<Axes: xlabel='category'>
```

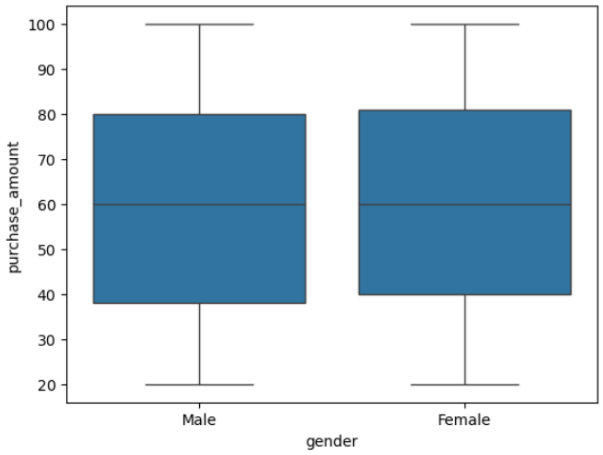



```
# 2.Customer Age Distribution
sns.histplot(df["age"])
```



```
#3. Gender vs Spending
sns.boxplot(x="gender", y="purchase_amount", data=df)
```

```
<Axes: xlabel='gender', ylabel='purchase_amount'>
```



7. SQL Analysis

Step 1: connect jupyter to SQL

```
# procedure to connect DataFrame to MySQL
!pip install sqlalchemy pymysql
from sqlalchemy import create_engine

engine = create_engine("mysql+pymysql://root:sushmiSa067$@localhost/customer_shopping_behaviour")
import pandas as pd

df.to_sql("customer_behavior", engine, if_exists="replace", index=False)
```

```
Requirement already satisfied: sqlalchemy in c:\users\sushmitha\anaconda3\lib\site-packages (2.0.43)
Requirement already satisfied: pymysql in c:\users\sushmitha\anaconda3\lib\site-packages (1.1.2)
Requirement already satisfied: greenlet>=1 in c:\users\sushmitha\anaconda3\lib\site-packages (from sqlalchemy) (3.2.4)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\sushmitha\anaconda3\lib\site-packages (from sqlalchemy) (4.15.0)
3900
```

```
# Procedure to check if the DataFrame is loaded to Mysql
df_check = pd.read_sql("SELECT * FROM customer_behavior", engine)
df_check.head()
```

Step 2: Basic queries

```sql
CREATE DATABASE customer_shopping_behaviour;
USE customer_shopping_behaviour;
-- Business questions to be solved
-- 1. What is the total revenue generated by male vs female customers
SELECT gender,
SUM(purchase_amount) AS total_revenue
FROM customer_behavior
GROUP BY gender;
```

Result Grid | Filter Rows:

gender	total_revenue
Male	157890
Female	75191

```sql
-- 2. Which customers used a discount but still spent more than the average purchase amount
SELECT customer_id,purchase_amount
FROM customer_behavior
WHERE  discount_applied = "Yes" AND purchase_amount >=(SELECT AVG(purchase_amount) FROM customer_behavior);
```

customer_id	purchase_amount
2	64
3	73
4	90
7	85
9	97
12	68
13	72
16	81
20	90
22	62
24	88
29	94
32	79
33	67

```sql
-- 3.Which are the top 5 products with the highest average review rating?
SELECT item_purchased,
ROUND(AVG(review_rating),2) AS avg_review_rating
FROM customer_behavior
GROUP BY item_purchased
ORDER BY AVG(review_rating) DESC
LIMIT 5;
```

	item_purchased	avg_review_rating
▶	Gloves	3.86
	Sandals	3.84
	Boots	3.82
	Hat	3.8
	Skirt	3.78

```sql
-- 4. Compare the average purchase amounts between standard and express shipping
SELECT shipping_type,
ROUND(AVG(purchase_amount),2) AS avg_purchase_amount
FROM customer_behavior
WHERE shipping_type IN("Standard","Express")
GROUP BY shipping_type;
```

	shipping_type	avg_purchase_amount
▶	Express	60.48
	Standard	58.46

```sql
-- 5. Do subscribed customers spend more ? compare average spend and total revenue between subscribers and non- subscribers
SELECT subscription_status,COUNT(customer_id) as total_customer, ROUND(avg(purchase_amount),2) as avg_spend,SUM(purchase_amount) as total_revenue
FROM customer_behavior
GROUP BY subscription_status
ORDER BY total_revenue,avg_spend DESC;
```

	subscription_status	total_customer	avg_spend	total_revenue
▶	Yes	1053	59.49	62645
	No	2847	59.87	170436

```sql
-- 6. Which 5 products have the highest percentage of purchases with discounts applied?
SELECT item_purchased,
ROUND(SUM(CASE WHEN discount_applied = "Yes"THEN 1 ELSE 0 END)/COUNT(*) ,2) AS discount_rate
FROM customer_behavior
GROUP BY item_purchased
ORDER BY discount_rate desc
limit 5;
```

item_purchased	discount_rate
Sneakers	0.50
Hat	0.50
Coat	0.49
Sweater	Coat
Pants	0.47

```sql
-- 7. Segment customers into New,Returning and loyal based on their total number of previous purchases, and show the count of each segment
WITH customer_type AS (
SELECT customer_id,previous_purchases,
CASE
    WHEN previous_purchases = 1 THEN "new"
    WHEN previous_purchases BETWEEN 2 AND 10 THEN "returning"
    ELSE "loyal"
    END AS customer_segment
FROM customer_behavior
)
SELECT customer_segment,COUNT(*) AS "number_0f_customers"
FROM customer_type
GROUP BY customer_segment;
```

customer_segment	number_0f_customers
loyal	3116
returning	701
new	83

```sql
-- 8. What are the top 3 most purchased products within each category?
WITH item_counts AS(
select category,
item_purchased,
COUNT(customer_id) AS total_orders,
ROW_NUMBER() OVER(PARTITION BY category ORDER BY COUNT(customer_id) DESC) AS item_rank
FROM customer_behavior
GROUP BY category,item_purchased
)
SELECT item_rank,category,item_purchased,total_orders
FROM item_counts
WHERE item_rank <=3;
```

	item_rank	category	item_purchased	total_orders
▶	1	Accessories	Jewelry	171
	2	Accessories	Sunglasses	161
	3	Accessories	Belt	161
	1	Clothing	Blouse	171
	2	Clothing	Pants	171
	3	Clothing	Shirt	169
	1	Footwear	Sandals	160
	2	Footwear	Shoes	150
	3	Footwear	Sneakers	145
	1	Outerwear	Jacket	163
	2	Outerwear	Coat	161

```sql
-- 9. Are customers who are repeat buyers (more than 5 previous purchases) also likely to subscribe
SELECT subscription_status,
COUNT(customer_id) AS repeat_buyers
FROM customer_behavior
WHERE previous_purchases > 5
GROUP BY subscription_status;
```

	subscription_status	repeat_buyers
▶	Yes	958
	No	2518

```sql
-- 10. What is the revenue contribution of each age group
SELECT age_group,
SUM(purchase_amount) AS revenue_contribution
FROM customer_behavior
GROUP BY age_group
ORDER BY revenue_contribution DESC;
```

	age_group	revenue_contribution
▶	Young Adult	62143
	Middle-aged	59197
	Adult	55978
	Senior	55763

8. Power BI Dashboard

Step 1: Load data from SQL database

Step 2: Create Measures (DAX)

- average_purchase_amount = AVERAGE(customer_shopping_behavior[purchase_amount])
- average_review_rating = AVERAGE(customer_shopping_behavior[review_rating])
- number_of_customers = COUNT(customer_shopping_behavior[customer_id])

Step 3: Create Visuals

- KPI Cards → No. of customers, Average purchase amount, average review rating
- Bar Chart → Category-wise - revenue, category-wise-sales
- Pie Chart → % of customers by subscription status
- Slicers → Category, Gender, shipping type, subscription status

9. Key Insights

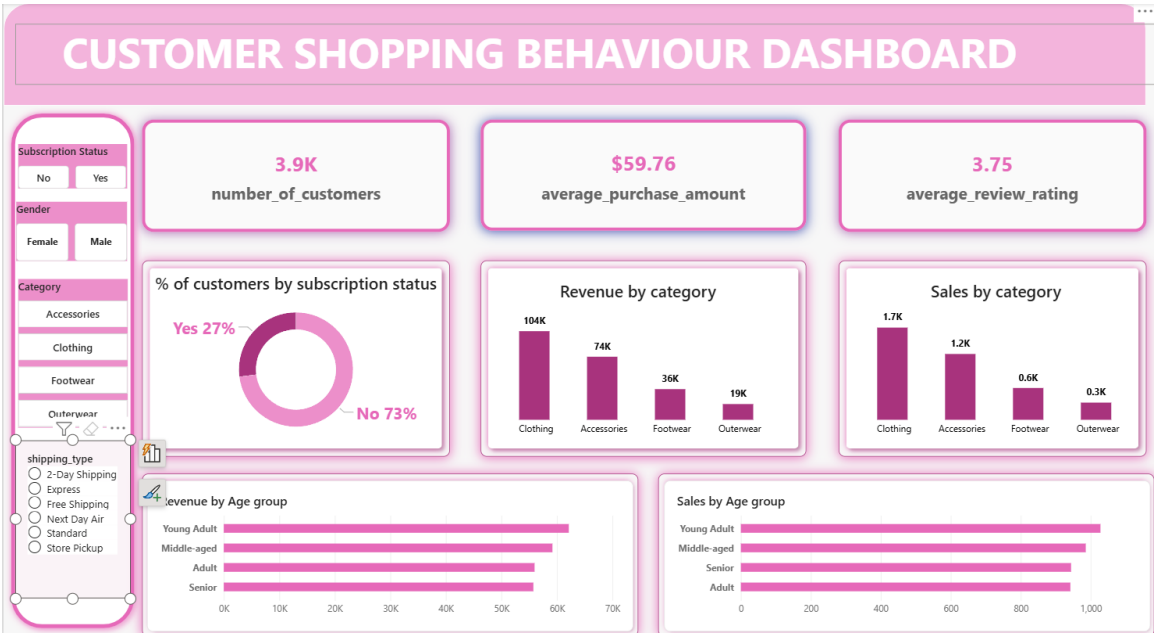
• Clothing category generates the highest revenue

• Also clothing category contributes most of the sales

• Seasonal trends affect purchasing behavior

•Male customers bring in more revenue

• Age influence spending patterns



10. Recommendations

Target high-value customers

Promote top products

Introduce loyalty programs

11. Conclusion

This project demonstrates how businesses can use data analytics to understand customer behavior, optimize product offerings, and improve decision-making.

12. Future Scope

Build ML models for recommendations

Use real-time data integration